

Architecture Overview

Components, tools, execution approach, control flow, and key decisions for the Spring Boot URL shortener

System Objective

The project is a production-ready modular monolith URL shortener. It exposes APIs to create short URLs, redirect short codes, collect click analytics, report health, and publish Swagger documentation. The runtime stack is Java 21, Spring Boot 3.x, PostgreSQL, Redis, Docker, and Google Cloud Run deployment assets.

Component Overview

Component	Responsibility	Main package or file
HTTP API	Receives create, redirect, analytics, health, and Swagger requests.	controller, Spring Actuator, SpringDoc
URL Service	Validates input, detects duplicates, generates short codes, resolves redirects, coordinates cache and analytics.	service, service.impl
Analytics Service	Persists click timestamp, browser, device, and referrer; returns aggregate and recent click data.	analytics
Persistence	Stores URL and Analytics entities with unique constraints for original URL and short code.	entity, repository
Redis Cache	Caches shortCode to originalUrl mappings and tolerates Redis read/write failures.	cache
Validation and Utilities	Enforces valid HTTP/HTTPS URLs and generates random Base62 short codes.	validation, util
API Mapping and Errors	Maps entities to immutable DTOs and returns structured error responses.	mapper, dto, exception

Tools and Runtime Stack

Area	Tooling
Language and framework	Java 21, Spring Boot 3.x, Spring MVC
Persistence	PostgreSQL, Spring Data JPA, Hibernate
Cache	Redis through Spring Data Redis
Validation	Spring Validation and custom URL validation
Observability and API docs	Spring Actuator health endpoint, SpringDoc OpenAPI Swagger UI
Build and packaging	Gradle, Docker multi-stage build
Local runtime	Docker Compose with app, PostgreSQL, and Redis
Testing	JUnit 5, Mockito, MockMvc, Testcontainers
Cloud deployment	Google Cloud Run, Artifact Registry, Cloud SQL, Memorystore, Secret Manager, Serverless VPC Access
CI/CD	GitHub Actions with Workload Identity Federation

Execution Approach

- ? Phase 1 established the Spring Boot Gradle project, configuration, application entry point, and JPA entities.
- ? Phase 2 implemented API controllers, service interfaces, service implementations, cache access, analytics, validation, mappers, DTOs, and global exception handling.
- ? Phase 3 added unit, controller, repository, and integration tests. Docker-backed tests are tagged as integration to isolate Testcontainers from fast local tests.

? Phase 4 added Docker, Docker Compose, Cloud Run deployment assets, GitHub Actions automation, and operational documentation.

Control Flow

Create short URL

```
Client -> POST /api/v1/urls
Controller -> UrlShortenerService.createShortUrl
UrlValidator -> accept only http/https URLs with host and no credentials
UrlRepository -> find duplicate originalUrl
ShortCodeGenerator -> create random Base62 shortCode
UrlRepository -> persist URL row in PostgreSQL
UrlCache -> write shortCode -> originalUrl in Redis
Controller -> return 201 Created or 200 OK for duplicate URL
```

Redirect short code

```
Client -> GET /{shortCode}
Controller -> UrlShortenerService.resolveOriginalUrl
UrlCache -> try Redis lookup
UrlRepository -> fallback to PostgreSQL and refresh Redis on cache miss
ClickAnalyticsService -> persist click event
Controller -> return 302 Found with Location header
```

Analytics lookup

```
Client -> GET /api/v1/analytics/{shortCode}
Controller -> ClickAnalyticsService.getAnalytics
UrlRepository -> find URL by shortCode
AnalyticsRepository -> count clicks and fetch recent clicks
AnalyticsMapper -> return analytics response DTO
```

Key Design Decisions

Decision	Why	Trade-off
Modular monolith	Keeps deployment simple while preserving package boundaries.	Modules are not independently deployable.
PostgreSQL unique constraints	Enforces duplicate URL and short-code integrity at the database layer.	Rare race conditions must be handled by service retry logic.
Redis read-through cache	Improves redirect latency and reduces database reads.	Analytics still requires the URL entity from PostgreSQL.
Random Base62 short codes	Avoids predictable sequential URLs.	Requires bounded collision retries.
Duplicate create returns 200 OK	Makes create idempotent for the same original URL.	Clients must distinguish 200 existing from 201 new.
Structured global errors	Keeps API failures consistent and avoids exposing stack traces.	Requires explicit exception mapping.
Hibernate ddl-auto update	Fast local and assignment setup.	Use Flyway or Liquibase for stricter production migrations.
Secret Manager for deployment	Avoids hardcoded database credentials.	Requires IAM setup for runtime and deploy service accounts.

Deployment Control Flow

```
Push to main -> GitHub Actions
Run gradle clean test -> Build Docker image
Push image to Artifact Registry
Deploy revision to Cloud Run
Cloud Run reads DB credentials from Secret Manager
Cloud Run connects to Cloud SQL PostgreSQL and Memorystore Redis
```

Operational Verification

? Health endpoint: GET /actuator/health returns UP.
? Create endpoint: POST /api/v1/urls returns shortCode and shortUrl.

- ? Redirect endpoint: GET /{shortCode} returns HTTP 302 and Location header.
- ? Analytics endpoint: GET /api/v1/analytics/{shortCode} returns persisted click count and recent clicks.
- ? Swagger UI: /swagger-ui.html loads generated OpenAPI documentation.

Package Boundary Summary

Package	Role
config	Application, auditing, and OpenAPI configuration.
controller	HTTP request and response handling.
dto	Immutable API payloads and service results.
entity	JPA entities for urls and analytics tables.
exception	Domain exceptions and global exception mapping.
mapper	Entity to DTO transformations.
repository	Spring Data JPA persistence interfaces.
service / service.impl	URL shortening use cases and orchestration.
cache	Redis integration.
analytics	Click capture and analytics querying.
validation	Original URL validation rules.
util	Short code generation and user-agent parsing.